Tầng A — pipeline chi tiết

công thức và ví dụ chạy tay

Mục lục

1 Ký hiệu **2**

2 Ví dụ dùng xuyên suốt **3**

3 Bước truy hồi — chung cả hai pha, 0 lượt LLM **3**

4 `A0` / `C0` — chẻ câu hỏi và bảng phủ **4**

4.1 1 Chẻ câu hỏi . . . 4

4.2 2 Bảng phủ . . . 4

4.3 3 Định nghĩa `adq` . . . 4

4.4 4 Cửa thoát . . . 5

4.5 5 Nghiệm phủ . . . 5

5 Hàm giá trị `v` **5**

6 Ngưỡng `_q` **6**

7 Điều kiện nhận — hai mệnh đề, không phải một **6**

8 Pha 1 — bốn vòng **7**

8.1 1 Vòng 0 — quét cố định, không cần `_q` . . . 7

8.2 2 Vòng 1 — greedy biên, có ràng buộc phủ . . . 7

8.3 3 Phân bổ `m` lượt cho các cụm . . . 8

8.4 4 `m` thích nghi . . . 8

8.5 5 Vòng 2 — active learning . . . 9

8.6 6 Vòng 3 — xác nhận bằng model thật . . . 9

8.7 7 Tổng chi phí pha 1 . . . 9

9 Tính tối tiểu — kiểm thế nào cho rẻ **9**

10 Pha 2 — đường rẻ **10**

10.1 1 Bảy bước . . . 10

10.2 2 Tìm theo tầng — đi lên từ nghiệm phủ . . . 11

10.3 3 Beam mò — nhánh dự bị . . . 11
10.4 4 c7 — ba nhánh . . . 12

11 Cứu — chạy pipeline đầy đủ **13**
11.1 1 Phân tầng . . . 13

12 Chỉ số cấu trúc và đầu ra **14**
12.1 1 Chọn T khi có nhiều MWC . . . 14
12.2 2 Đầu ra . . . 14

13 Phép kiểm và chuẩn ngoài **15**
13.1 1 Phép kiểm nội bộ — năm điều . . . 15
13.2 2 Chuẩn ngoài — HotpotQA . . . 15
13.3 3 Phân tích độ nhạy . . . 15

14 Chạy theo lô **16**

15 Sổ chi phí và sổ dữ liệu **16**

16 Chạy tay trọn vẹn một truy vấn — pha 2 **17**

17 Những con số chưa được kiểm chứng **18**

18 Tham chiếu **18**

Tài liệu này là bản **thi hành** của tầng A: từng bước, từng công thức, và một ví dụ số chạy xuyên suốt. Phần *vì sao* nằm ở tài liệu chính §3; ở đây chỉ nói *làm thế nào*.

Mọi tham chiếu §x.y trỏ sang tài liệu chính.

---

1. Ký hiệu

```
q               truy vấn
y               đáp án vàng - BẮT BUỘC có, nên tầng A chỉ chạy offline
C(q)            pool ứng viên sau rerank,  n = |C(q)| = 40
c = 1..C        chỉ số cụm,  C <= floor(m/2)
S c= C(q)        một tập con ứng viên
T               tập đích cuối cùng - THỨ DUY NHẤT đi sang tầng B

v(S)            điểm judge - model đọc S, trả lời, so với y          thuộc [0,1]
v_ref(q)        trần: điểm cao nhất pool này đạt được
tau_q             ngưỡng đủ
mảnh_j          sub-claim thứ j - câu hỏi chẻ thành khẳng định 1-hop
adq(S)          S có phủ hết mảnh không                              thuộc {đủ, không}
g_theta             surrogate, đoán v(S)                                  §3.5
m               số ứng viên chấm mỗi vòng greedy
r               SỐ VÒNG greedy - kết quả, không phải núm vặn
beta_i, PGI_i       Banzhaf có dấu; số MWC chứa i                        §3.7
```

---

2. Ví dụ dùng xuyên suốt

```
q  =  "ai đạo diễn phim thắng Oscar phim hay nhất 1994?"
y  =  "Robert Zemeckis"
```

Pool sau truy hồi và rerank, `n = 40`. Sáu đoạn đáng kể:

```
đoạn  3   "Forrest Gump thắng giải Phim hay nhất cho năm 1994."
đoạn  7   "Giải Phim hay nhất năm 1994 thuộc về Forrest Gump."        ~ trùng đoạn 3
đoạn 12   "Forrest Gump do Robert Zemeckis đạo diễn."
đoạn 23   "Robert Zemeckis là đạo diễn của Forrest Gump và Back to the Future."
đoạn 31   "Zemeckis đạo diễn Forrest Gump năm 1994."
đoạn 19   "Forrest Gump là một hiện tượng văn hóa thập niên 1990."    <- KHÔNG có dữ kiện
34 đoạn còn lại - nhiễu
```

Đoạn 19 là cái bẫy: nó không chứa dữ kiện nào, nhưng nhắc tên phim nên gợi model nhớ ra Zemeckis từ tri thức tham số.

---

3. Bước truy hồi — chung cả hai pha, 0 lượt LLM

```
3 nguồn  ->  pool thô  K <= 60  ->  khử trùng lặp
                 v  rerank THEO TỪNG NGUỒN, hạn ngạch riêng
n = 40   =   20 BM25   +   15 GFM   +   5 ngẫu nhiên
                 v
gom C cụm theo embedding (pha 1) hoặc theo phần dư của g_theta (pha 2)
```

Pool lấy từ node `type == "document"` của `processed/stage1/nodes.csv` (và `summary` khi có). `entity` không phải đơn vị văn bản nên không bao giờ vào prompt; `attribute` là cột, không phải loại node.

Đơn vị chunk do công cụ quyết định: `documents.json` là `{ "tên tài liệu": "nội dung" }` và mỗi entry là một document node — framework không cắt bên trong. Với HotpotQA / MuSiQue / 2Wiki thì đơn vị là đoạn Wikipedia có sẵn. Index BM25 và bộ bốc ngẫu nhiên dựng trên **đúng** tập node đó, nên tính đồng nhất cách cắt là đúng theo cấu trúc.

Trần cứng của `C`:

```
C  <=  floor( m / 2 )              m = 12 -> C <= 6;   m = 30 -> C <= 15
```

Lý do: mỗi cụm có **sàn** 1 lượt, nên sàn tiêu hết `C` trong ngân sách `m`. `C = m` thì phần bốc theo xác suất biến mất và phân bổ thoái hóa thành chia đều.

Chọn `C` trong trần đó:

```
pha 1    C  =  argmax silhouette trên embedding pool,   2 <= C <= floor(m/2)
pha 2    gom theo PHẦN DƯ của g_theta, không theo cosine:

             d(k, k')  =  g_theta(q,{k}) + g_theta(q,{k'}) - g_theta(q,{k,k'})

             d lớn  ->  trùng lặp  ->  CÙNG cụm
             d âm   ->  bắc cầu    ->  KHÁC cụm
         780 cặp cho 40 chunk, mỗi cặp một lượt forward mô hình nhỏ. 0 lượt LLM.
```

Trong ví dụ: `C = 6`, và các đoạn 3, 7 rơi vào cùng một cụm (trùng ý); 12, 23, 31 vào cụm khác.

4. `A0` / `C0` — chẻ câu hỏi và bảng phủ

4.1 1 Chẻ câu hỏi

Hỏi ai: `MuSiQue` gán sẵn. Corpus riêng thì gọi LLM **một lần cho mỗi truy vấn**, không phải mỗi tập con.

```
vào    q + y
ra     mảnh 1   "phim thắng Oscar phim hay nhất 1994 là Forrest Gump"
       mảnh 2   "Forrest Gump do Robert Zemeckis đạo diễn"
giá    ~400 token
```

Số mảnh `J` cho **prior về** `|T|`: `|T|` nên bằng `J`.

4.2 2 Bảng phủ

Hỏi ai: model NLI nhỏ (`~400M`), chạy local, **0 lượt API**.

```
cho mỗi đoạn k thuộc C(q),  cho mỗi mảnh j:
     e(k, j)  =  NLI( premise = đoạn k,  hypothesis = mảnh j )  thuộc {entail, neutral, contra}
```

`40 × 2 = 80` lượt forward mô hình nhỏ. Kết quả:

```
            mảnh 1      mảnh 2
đoạn  3     entail        -
đoạn  7     entail        -
đoạn 12       -         entail
đoạn 23       -         entail
đoạn 31       -         entail
đoạn 19       -           -          <- cái bẫy, không phủ mảnh nào
34 đoạn còn lại, không phủ gì
```

4.3 3 Định nghĩa `adq`

```
cov(S)  =  { j :  có k thuộc S,  e(k,j) = entail }

adq(S) = đủ   <=>   cov(S) = { 1, ..., J }
```

Ví dụ:

```
adq({3, 12})      cov = {1,2}  =  đủ
adq({19})         cov = rỗng      =  KHÔNG đủ        <- bẫy bị bắt ở đây
adq({3, 7})       cov = {1}    =  KHÔNG đủ        <- hai đoạn trùng ý, vẫn thiếu mảnh 2
adq({3,12,19})    cov = {1,2}  =  đủ              (19 là đoạn thừa)
```

4.4 4 Cửa thoát

```
có j :  không đoạn nào trong C(q) entail mảnh j
   ->  pool KHÔNG chứa tập đủ  ->  bỏ truy vấn, cờ "pool thiếu"
   ->  đã tiêu 400 token (không phải 127.500 ở pha 1, hay 21.900 ở pha 2)
```

Đặt **thận trọng**: chỉ thoát khi điểm `entail` của mọi đoạn đối mảnh `j` đều rất thấp, không phải chỉ khi nhãn là `neutral`. NLI hay bỏ sót bằng chứng diễn đạt lại.

Ca bị bỏ **không xóa** — nó là cờ `"pool thiếu"`, nguồn nuôi ô «lỗi đồ thị» §6.1 và vòng lặp ngược §6.6.

4.5 5 Nghiệm phủ

Bài toán phủ tập nhỏ, giải **chính xác**:

```
mảnh 1  <-  {3, 7}
mảnh 2  <-  {12, 23, 31}

nghiệm phủ tối tiểu:  {3,12} {3,23} {3,31} {7,12} {7,23} {7,31}      6 ứng viên, cỡ 2
```

Đây là điểm khởi cho §3.4 (greedy) và §3.6 (tìm theo tầng).

---

5. Hàm giá trị `v`

```
sinh     â_S   =  generate( q, S )                          ~80 token
chấm     v(S)  =  P_judge( "đúng" | q, â_S, y )  thuộc [0,1]
                  đọc logprob token đầu, KHÔNG decode câu trả lời judge
v(rỗng)           =  điểm khi không đoạn nào - chỉ tri thức tham số
```

Judge phải trả số liên tục, không phải thang 1–5. Thang 5 mức làm hàng chục tập con hòa nhau, `S` và `S {c}` không phân biệt được, và "tối tiểu" mất nghĩa.

Chi phí một lượt:

```
cost(S)  ~  500  +  250*|S|      token

v(rỗng)              =    500
v({k})             =    750
v(R),  |R| = 12    =  3.500
v(C),  |C| = 40    = 10.500
```

Các lượt **không bằng giá nhau** — đếm theo số lượt sẽ ước lượng sai hẳn.

Ví dụ, các giá trị đo được:

```
v(rỗng)         = 0,05        model không biết
v({19})      = 0,71        <- CAO, nhờ nhớ sẵn.  adq = KHÔNG đủ
v({3})       = 0,12
v({12})      = 0,18
v({3,7})     = 0,13        trùng ý, không cộng dồn
```

```
v({3,12})    = 0,94        bắc cầu
v({3,12,19}) = 0,95
v(R)         = 0,88
v(C)         = 0,81        phân tán vì nhồi 40 đoạn
```

---

6. Ngưỡng `_q`

```
v_ref(q)  =  max( v(C),  v(R),  max_k v({k}),  mọi v(S) đã quan sát )

tau_q  =  theta * [ v_ref(q) − v(rỗng) ]  +  v(rỗng)              theta = 0,95
```

Hai định nghĩa theo pha:

```
pha 1   v_ref = max( v(C), v(R), max_k v({k}), mọi v(S) trong greedy )
pha 2   v_ref = max( v(R), v(C) )                 pha 2 không quan sát tập con nào
```

Chốt chặn mẫu số nhỏ:

```
v_ref − v(rỗng)  <  0,05    ->  T = rỗng,  cờ "truy hồi không đóng góp"
```

Mốc phải là cực đại quan sát được, không phải `v(C)`**.** Khi `C` còn đoạn nhiễu thì một tập con có thể ghi điểm cao hơn cả `C`; neo vào `v(C)` là neo vào mốc bị dìm.

Trong ví dụ:

```
v_ref  =  max( 0,81 ; 0,88 ; 0,71 )  =  0,88        <- v(R) thắng v(C), đúng như cảnh báo
tau_q    =  0,95 x (0,88 − 0,05) + 0,05  =  0,8385   ->  0,84
```

Tính tự nhất quán của cập nhật. Nếu quan sát mới `v(S) > v_ref` thì `v_ref := v(S)` và

```
tau_q'  =  theta*[v(S) − v(rỗng)] + v(rỗng)  <  v(S)        vì theta = 0,95 < 1
```

nên tập vừa nâng trần **luôn vẫn vượt ngưỡng mới**. Cập nhật không bao giờ tự phủ định cái gây ra nó, và vì `_q` chỉ tăng, mọi tập đã bị loại vẫn bị loại. `< 1` gánh toàn bộ tính chất này.

---

7. Điều kiện nhận — hai mệnh đề, không phải một

```
S đủ          v(S) >= tau_q          VÀ     adq(S) = đủ
S tối tiểu    đủ, và không tập con thực sự nào của S còn đủ

T   =   một S đủ tối tiểu
```

Kiểm trên ví dụ:

```
S = {19}       v = 0,71 < 0,84        trượt v.   (và adq cũng trượt)
S = {3,7}      v = 0,13 < 0,84        trượt cả hai
S = {3,12}     v = 0,94 >= 0,84       adq = đủ       tối tiểu?  xem §8
S = {3,12,19}  v = 0,95 >= 0,84       adq = đủ       NHƯNG {3,12} c nó cũng đủ
                                                      ->  KHÔNG tối tiểu
```

Bảng 2×2 — đọc kết quả cho đúng:

```
                     adq đủ                        adq KHÔNG đủ
v >= tau_q        nhãn TỐT - nhận               MỒI THAM SỐ - loại
v <  tau_q       bằng chứng CÓ mà model        pool THIẾU - thoát sớm ở A0/C0
                 không nối được
                 -> lỗi MODEL SINH
```

Ô dưới-trái là ô mà bản chỉ-dùng-v không tách được. Nếu nó hay xảy ra thì **tầng B không sửa được** — phải đổi model sinh hoặc đổi prompt.

---

8. Pha 1 — bốn vòng

8.1 1 Vòng 0 — quét cố định, không cần _q

```
v(rỗng)                                    500
v(R)     R = 12 đoạn, cách chọn ở dưới   3.500
v(C)     cả 40 đoạn                     10.500
v({k})   cả 40 đoạn đơn                 30.000
                                        -------
                                        44.500 token
->  v_ref,  tau_q  chốt tại đây
```

Chọn R: sàn cụm thắng hạn ngạch nguồn. Hai ràng buộc xung đột thường xuyên vì cụm gom theo embedding chứ không theo nguồn.

```
6 suất   mỗi cụm 1 đoạn, lấy đoạn hạng cao nhất trong cụm      SÀN CỤM - bắt buộc
6 suất   còn lại: trước hết bảo đảm mỗi nguồn có >= 1 trong R
                  rồi lấp phần dư theo hạng
                                                               |R| = 12
```

Sàn cụm thắng vì nhiệm vụ của v(R) là đo **trần**, mà trần phải chạm *mọi* chủ đề. Và 2 đoạn mỗi cụm chứ không 1 — với 1 thì một **cặp bắc cầu nằm trong cùng cụm** không bao giờ hiện ra.

8.2 2 Vòng 1 — greedy biên, có ràng buộc phủ

```
S_0 = rỗng,  cov_0 = rỗng
mỗi vòng t:
    ứng viên hợp lệ  =  { k không thuộc S_t :  k phủ được một mảnh CHƯA nằm trong cov_t }
    shortlist m ứng viên theo phân bổ cụm (§7.3)
    đo  v(S_t + {k})  cho cả m ứng viên,  GHI CẢ m, kể cả cái bị loại
    k*  =  argmax_k  v(S_t + {k})
```

```
    S_{t+1} = S_t + {k*}

dừng khi   v(S_t) >= tau_q   VÀ   cov_t = mọi mảnh
```

Ba chỗ khác bản cũ:

```
HƯỚNG ĐI   phủ là ràng buộc CỨNG, v là mục tiêu TRONG ràng buộc đó
DỪNG       thêm điều kiện phủ.  v >= tau_q mà chưa phủ hết = ca mồi tham số -> ĐI TIẾP
pi_c        lấy từ bảng phủ, không từ singleton
```

Vòng greedy thứ nhất miễn phí — nó chính là chấm singleton mà vòng 0 đã làm. Nên chi phí là `(r − 1)·m`, không phải `r·m`.

Chạy trên ví dụ:

```
vòng 1   cov = rỗng.  ứng viên hợp lệ: mọi đoạn phủ mảnh 1 hoặc 2 = {3,7,12,23,31}
         (đoạn 19 KHÔNG hợp lệ - nó không phủ mảnh nào)
         đo: v({3}) = 0,12 * v({7}) = 0,11 * v({12}) = 0,18 * v({23}) = 0,16 * v({31}) = 0,15
         k* = 12.   S_1 = {12},  cov_1 = {2}
vòng 2   ứng viên hợp lệ: chỉ đoạn phủ MẢNH 1 = {3, 7}
         đo: v({12,3}) = 0,94 * v({12,7}) = 0,92
         k* = 3.    S_2 = {3,12},  cov_2 = {1,2}
dừng     v = 0,94 >= 0,84      cov đầy           r = 2
```

So với bản cũ: bản cũ sẽ thử cả đoạn 19 và thấy `v({19}) = 0,71`, cao hơn mọi singleton hợp lệ — rồi có thể đi theo nhánh đó. Ràng buộc phủ loại nó khỏi danh sách ngay từ đầu.

8.3 3 Phân bổ `m` lượt cho các cụm

```
tiềm năng của cụm c tại vòng t:

    pi_c  =  max_{k thuộc c \ S_t,  k hợp lệ}   gˆain( k | S_t )

    gˆain   vòng 2:   v({k}) - v(rỗng)                  singleton đã có sẵn
           vòng 3+:  g_theta(S_t + {k}) - g_theta(S_t)      khi g_theta dùng được
           và ưu tiên k phủ mảnh CÒN THIẾU           từ bảng phủ

phân bổ:

    a_c  =  1  +  Multinomial( m − C ;  softmax(pi_c / T) )_c
            ---               ------------------------------
            sàn, MỌI cụm      phần dư chia theo xác suất
```

`C = 6, m = 12`: sàn tiêu 6, còn 6 lượt bốc theo . Cụm nóng nhận `1+4 = 5`, cụm chết nhận `1`. **Tổng vẫn đúng `m`.**

Bốc xác suất chứ không chia tỉ lệ tất định, vì chia tất định thì một cụm bị đánh giá thấp sẽ **không bao giờ** được thăm dò lại.

8.4 4 `m` thích nghi

```
spread  =  max_k v({k}) - v(rỗng)
```

```
spread >= 0,15 * [ v_ref − v(rỗng) ]                 ->  m = 12
spread <  ngưỡng đó  VÀ  bảng phủ cũng mù        ->  m = 30
spread <  ngưỡng đó  mà bảng phủ CÓ tín hiệu     ->  m = 12
```

Dòng thứ ba là chỗ bảng phủ tiết kiệm: ở multi-hop **mọi** singleton `v( )` nên `spread` thấp và `_c` là nhiễu — đó là lý do bản cũ phải nâng `m` lên 30. Bảng phủ nói đoạn nào mang bằng chứng kể cả khi không singleton nào làm `v` nhích.

```
m      lượt    token greedy    tổng mỗi truy vấn
12      24        27.000          128.000
30      60        67.500          168.500
```

Tiết kiệm `40.500` token mỗi truy vấn multi-hop.

8.5 5 Vòng 2 — active learning

```
~30 lượt mỗi truy vấn, lặp 2-3 vòng
chọn mẫu thỏa CẢ HAI:
    ensemble g_theta bất đồng nhiều
    dự đoán rơi quanh tau_q
```

8.6 6 Vòng 3 — xác nhận bằng model thật

```
v(T) >= tau_q                     1 lượt
adq(T) = đủ                     0 lượt  (bảng phủ đã có)
tập con còn phủ hết < tau_q       xem §8
```

Không bao giờ để proxy quyết định nhận hay bỏ.

8.7 7 Tổng chi phí pha 1

```
A0  chẻ + bảng phủ                      400
A1  vòng 0                            44.500
A2  vòng 1,  m = 12,  r = 2-3         27.000
A3  vòng 2                           ~37.000
A4  vòng 3                           ~15.000
                                     --------
                                     123.900 token  ~  $0,37    ở $3/triệu
```

Nếu `A0` thoát: **400 token**, không phải 123.900.

---

9. Tính tối tiểu — kiểm thế nào cho rẻ

Phải quét hết tập con, vì `v` **không đơn điệu**:

```
v(S bỏ {c}) < tau_q   KHÔNG kéo theo   v(tập con nhỏ hơn) < tau_q
```

Ca phản ví dụ, `b` là đoạn gây nhiễu:

```
tau_q = 0,76
v({a})     = 0,85  >= tau_q      ->  {a} MỘT MÌNH đã đủ
v({a,b})   = 0,60  < tau_q
v({a,c})   = 0,71  < tau_q
v({b,c})   = 0,40  < tau_q
v({a,b,c}) = 0,80  >= tau_q

bỏ-một-phần-tử:  cả ba cặp trượt  ->  tuyên "tối tiểu"      SAI
sự thật:         {a} thắng        ->  thừa cả b và c
```

Nhưng bảng phủ làm phép quét co lại rất mạnh. Vì điều kiện nhận có hai mệnh đề, một tập con chỉ **đe dọa** tính tối tiểu nếu nó **cũng phủ hết**:

```
T = {3, 12, 19}      3 phủ mảnh 1 * 12 phủ mảnh 2 * 19 thừa

tập con thực sự:            7 tập
trong đó CÒN phủ hết:       1 tập  -  {3,12}
     ({3,19} mất mảnh 2 * {12,19} mất mảnh 1 * mọi đơn tử mất một mảnh)

->  chỉ cần kiểm {3,12}
```

Và nếu tìm theo tầng (§9) thì `{3,12}` **đã được đo ở tầng 0** — nên tính tối tiểu không tốn lượt nào thêm.

Chi phí quét hết khi **không** có bảng phủ, để so:

```
|S*|   lượt kiểm      token      tiền
 2      2 + 1 = 3      3.000    $0,009
 3      6 + 1 = 7      6.500    $0,020
 4     14 + 1 = 15    15.500    $0,047
 5     30 + 1 = 31    35.500    $0,107
 6     62 + 1 = 63    79.500    $0,239
 8    254 + 1 = 255  381.000    $1,14
10   1022 + 1 = 1023 1.788.500  $5,37
```

Nổ theo `2^s`. Đó là lý do nhánh **beam mò dự bị** (§9.3) vẫn cần ba ngưỡng và cơ chế tỉa.

---

10. Pha 2 — đường rẻ

10.1 1 Bảy bước

```
C0  chẻ câu hỏi + bảng phủ (§3)                          400 token
      thiếu mảnh  ->  THOÁT, cờ "pool thiếu"
C1  v(rỗng)                                                 500 token
C2  g_theta chấm 40 đoạn đơn,  v~_k che                          0 token
C3  v(R)                                                  3.500 token
C4  v(C)                                                 10.500 token
      -- v_ref = max( v(R), v(C) ) ;  tau_q chốt --
      v_ref - v(rỗng) < 0,05  ->  T = rỗng, cờ "truy hồi không đóng góp"
C5  tìm theo tầng từ nghiệm phủ (§9.2)                       0 token
```

```
      mọi tầng trượt  ->  beam mò dự bị (§9.3)
C6  verification bằng LLM THẬT                          ~7.400 token
C7  ba nhánh (§9.4)
                                                       -------------
                                                        ~22.300 token  ~ $0,066
```

C2 phải đứng trước C5 vì tìm theo tầng cần điểm từng đoạn để xếp hạng; hợp lệ vì g_ **không nhận** _q (§3.5).

Kỳ vọng mỗi truy vấn **nạp vào**, nếu ~45% thoát ở C0:

```
0,45 x 400  +  0,55 x 22.300   =   12.400 token   ~  $0,037
```

10.2 2 Tìm theo tầng — đi lên từ nghiệm phủ

```
tầng 0   các nghiệm phủ tối tiểu              |T| = J
tầng 1   nghiệm phủ + 1 đoạn
tầng 2   nghiệm phủ + 2 đoạn
...
mỗi tầng:  chấm bằng g_theta, lấy top-beam, đo v theo thứ tự đó
```

Vì sao chỉ tìm trong vùng này:

```
tập KHÔNG phủ hết  ->  nếu vượt tau_q thì chỉ có thể vì mỗi tham số
                   ->  mà ta đã quyết LOẠI ca đó
                   ->  tìm ở đó là tìm thứ ta sẽ bỏ
```

Chạy trên ví dụ:

```
tầng 0   6 nghiệm: {3,12} {3,23} {3,31} {7,12} {7,23} {7,31}
         g_theta xếp hạng -> {3,12} đứng đầu
         đo v({3,12}) = 0,94 >= 0,84   ->  QUA ngay ở tầng 0
         tối tiểu: tập con còn phủ hết = rỗng (cỡ 2 là cỡ tối tiểu của phủ)
                   -> chỉ cần kiểm hai đơn tử: v({3}) = 0,12 * v({12}) = 0,18, cả hai < tau_q
         T = {3, 12}
```

Hai lợi ích, cả hai rơi ra từ cấu trúc:

```
1  MỌI ứng viên đã đủ về bằng chứng - không còn ứng viên mỗi tham số
2  thứ tự theo CỠ -> "tối tiểu trước" miễn phí, không cần bước tỉa riêng
```

Giới hạn thật: một tầng có nhiều ứng viên hơn mức đo nổi (tầng 1 với n = 40 và 6 nghiệm phủ có tới ~228 tập). Nên chỉ đo top vài cái theo g_ — tính tối tiểu là *"tối tiểu trong số đã thử"*, không phải tối tiểu tuyệt đối. T ở pha 2 là một MWC **trong** vùng đã thử, không phải MWC toàn cục của C.

10.3 3 Beam mò — nhánh dự bị

Dùng khi mọi tầng ở §9.2 đều trượt (bảng phủ thiếu cột vì NLI bỏ sót bằng chứng diễn đạt lại):

```
frontier = { rỗng }
cho size = 1, 2, ..., n:
    ứng viên = { S + {i} : S thuộc frontier, i không thuộc S }
    chấm bằng g_theta
    đủ = { S : g_theta(S) >= tau_q }
    nếu đủ khác rỗng:  thu hoạch, và chạy tiếp thêm 1-2 cỡ nữa
    frontier = top-beam theo g_theta

tỉa:  bỏ c thuộc S nếu g_theta(S bỏ {c}) >= tau_q, lặp tới khi không bỏ được
      thứ tự bỏ: theo r_i tăng dần (đoạn rerank xếp thấp bị thử bỏ trước)
```

Tỉa **phụ thuộc thứ tự** vì v không đơn điệu — chốt một thứ tự và ghi rõ.
Ở nhánh này ba ngưỡng |S*| vẫn cần, vì không có bảng phủ để co phép quét:

```
|S*| <= 5          quét hết ngay                        $0,02 - $0,11
6 <= |S*| <= 10     tỉa xuống <= 5 rồi quét hết           $0,04 - $0,10
|S*| >= 11         cứu                                  $0,34
tỉa KẸT ở >= 6     cứu                                  $0,34
```

Tỉa rồi quét **vẫn là chứng minh đầy đủ**: §6 chỉ đòi T *là một* MWC, không đòi duy nhất hay nhỏ nhất.

10.4 4 C7 — ba nhánh

```
(1) v(T) >= tau_q  và  adq(T) = đủ  và  tối tiểu
        ->  xuất T sang tầng B

(2) v(T) < tau_q                        T KHÔNG đủ - g_theta đoán cao quá
        ->  thử ứng viên kế, tối đa 3 lần
            mỗi lần thử chỉ 1 lượt (hết đủ thì dừng, không kiểm tối tiểu)
        ->  qua -> nhánh (1);   3 lần đều trượt -> nhánh (3)

(3) |T| >= 11,  tỉa kẹt ở >= 6,  hoặc 3 lần thử đều trượt
        ->  hàng cứu có phân tầng (§10)
```

Ca "đủ mà không tối tiểu" không vào hàng cứu. Nếu quét hết thì cả dàn dưới T đã có giá trị đo, nên phần tử tối tiểu đọc ra ngay, 0 lượt thêm:

```
v({3,12,19}) = 0,95 >= tau_q        đủ
v({3,12})    = 0,94 >= tau_q        tập con cũng đủ  ->  KHÔNG tối tiểu
v({3}) = 0,12   v({12}) = 0,18   cả hai dưới ngưỡng
->  T = {3,12}
```

Đây là ca xảy ra thường xuyên, vì lỗi hay gặp nhất của g_ là đề xuất tập **phình**.

Nhánh (2) rẻ hơn tưởng: hết đủ thì dừng ngay nên một ca trượt chỉ tốn 1 lượt (1.250 token), còn một lượt cứu tốn 113.000 — **mua được 90 lần thử bằng giá một lần cứu**. Dừng ở 3 vì ứng viên sau cùng một mô hình nên cùng thiên lệch. Với tỷ lệ qua có điều kiện ~30%:

```
p hiệu dụng   0,75  ->  0,75 + 0,25 x 0,66  =  0,91
```

Phải **ghi lại hạng ứng viên được nhận**: nhận ứng viên #3 đẩy phân bố `v(T) - _q` xuống sát ngưỡng hơn, và tầng B cần biết để đánh trọng số.

---

11. Cứu — chạy pipeline đầy đủ

Cứu = chạy `A1`–`A4` lên truy vấn đó. Nó không bắt đầu từ 0:

```
đã có trong cache   v(rỗng) 500 * v(R) 3.500 * v(C) 10.500          14.500
                    + v(T) và dàn tập con đã đo ở C6
còn phải chạy       40 singleton                                    30.000
                    vòng 1 greedy biên                              27.000
                    vòng 2 active learning                          37.000
                    vòng 3 xác nhận                                  6.500
                                                                  ---------
                    cứu, tính biên                                 113.000 token  ~ $0,34
```

`_q` có thể TĂNG khi cứu, và đó là đúng. Cứu quan sát thêm 40 singleton và hàng chục tập con greedy, nên `v_ref` lấy max trên tập quan sát lớn hơn. `v_ref` chỉ có thể lên, nên `_q` lên theo: một truy vấn trượt ngưỡng pha 2 có thể gặp ngưỡng **cao hơn** lúc được cứu. Pha 2 đang đo trần thiếu, và cứu sửa chỗ đó.

Cứu mà vẫn trượt là ca có giá trị nhất:

```
greedy chạy hết, active learning xong, không tập nào vượt tau_q
->  T = rỗng  +  cờ "pool không chứa tập đủ"
->  nuôi ô "lỗi đồ thị" §6.1 và vòng lặp ngược §6.6
```

11.1 1 Phân tầng

Cứu hết thì `~17.000` ca × `$0,34` = `$5.780` để lấy `17.000` nhãn. Nhưng tính theo **mẫu huấn luyện** thì ngược:

```
cứu              111 mẫu / $0,34    =   $0,0031 / mẫu
đường rẻ, qua     10 mẫu / $0,066   =   $0,0066 / mẫu
```

Hai mục đích muốn hai cách lấy mẫu ngược nhau:

```
ĐO     "bao nhiêu % ca trượt là g_theta sai, bao nhiêu là pool thiếu?"
       -> mẫu KHÔNG LỆCH: ngẫu nhiên trong từng tầng, kèm trọng số N_s / n_s
TRAIN  nuôi g_theta đúng chỗ nó yếu
       -> càng nhiều càng tốt, lệch về tầng khó nhất
```

Cách chia: hạn mức nhỏ cố định cho **đo** (ví dụ 50 ca mỗi tầng), phần còn lại dồn cho tầng khó để **train**.

```
tầng                          ý nghĩa                          ưu tiên
trượt, hỏng ở ứng viên #1     g_theta lệch mạnh                    cao
trượt, hỏng cả 3 ứng viên     g_theta sai hệ thống                 CAO NHẤT
tỉa kẹt ở |S| >= 6             sai cả hai chiều                  CAO NHẤT
```

```
|T| >= 11                      phình nặng                        cao
nhiều hop                     ca bắc cầu - đúng thứ ta cần      CAO NHẤT
v(T) sát ngay dưới tau_q        có thể chỉ là trần đo thiếu       thấp
phiên bản g_theta^(b)             để tách xem lô nào yếu            (chỉ để đo)
```

Trọng số importance — một chỗ dễ dùng sai:

```
ĐO     -> PHẢI dùng trọng số (chỉ cứu 10-40%, mẫu lệch)
TRAIN  -> ĐỪNG hạ trọng số ca leo thang; chúng là hard example, muốn chúng NẶNG
```

Dùng chung một bộ trọng số cho cả hai là lỗi âm thầm: model vẫn train, chỉ là nó học nhẹ đi đúng ở chỗ nó yếu.

Ca bị bỏ vẫn không vô giá trị: mỗi ca trượt không được cứu vẫn xuất `v( )`, `v(R)`, `v(C)`, `v(T)` và dàn tập con đã đo — 4+ nhãn thật — cộng sự kiện *"g_ đề xuất T mà LLM bác"*. Ghi vào log train; chỉ **không** xuất `T`.

---

12. Chỉ số cấu trúc và đầu ra

```
beta_i+     số lần thêm i biến thiếu -> đủ
beta_i-     số lần thêm i biến đủ -> thiếu              đơn điệu thì bằng 0
beta_i   =  ( beta_i+ - beta_i- ) / 2^(n-1)                  ước lượng bằng lấy mẫu trên g_theta

PGI_i    số MWC trong W có chứa i

overlap(S)  =  tổng_{S' != S}  |S ^ S'| / |S + S'|
```

Chuẩn hóa và `PGI` riêng rồi mới gộp. `[-1,1]` còn `PGI` là số đếm có thể tới hàng chục; cộng thẳng thì `PGI` nuốt một đến hai bậc độ lớn.

`_i < 0` **chỉ thẳng ra đoạn gây hại.** Tính không đơn điệu ở đây là thông tin, không phải khuyết tật.

12.1 1 Chọn `T` khi có nhiều MWC

```
parsimony   - |S|
structure   sigma_g  =  w_beta * betâ  +  w_PGI * PGĨ        hai thành phần chuẩn hóa riêng
rank        tổng_i r_i                                  hạng rerank, chỉ phá hòa ở bước cuối

lọc Pareto, rồi gộp có trọng số
```

12.2 2 Đầu ra

```
T                          tập đích - CHỈ cái này đi qua ranh giới hai tầng
+ phiên bản g_theta            g_theta^(1) ... g_theta^(10),  lô nào gán
+ đường đi                 qua trực tiếp * giải trong dàn * tỉa * thử lại #k * được cứu
+ trọng số importance      N_s / n_s của tầng lấy mẫu
+ v(T) - tau_q               khoảng vượt ngưỡng
+ cov(T), J                phủ mấy mảnh trên mấy mảnh

W, p(S), entropy(p)        báo cáo và độ tin cậy
beta_i < 0                    ứng viên V_neg cho L_hop §6.3 - LẤY TỪ PHA 1, không từ pha 2
```

Entropy của `p(S)` **là tín hiệu độ tin cậy.** Thấp nghĩa là có một người thắng rõ ràng; cao nghĩa là nhiều MWC suýt soát và quy gán đang không chắc.

`W` và ở pha 2 **yếu hơn** pha 1, vì verification chỉ kiểm `T`, không kiểm cả họ `W`.

---

13. Phép kiểm và chuẩn ngoài

13.1 1 Phép kiểm nội bộ — năm điều

```
1  v(T) >= tau_q                                    đo bằng model THẬT, không phải g_theta
2  adq(T) = phủ hết mảnh                          bằng chứng thật sự nằm trong T
3  v(S') < tau_q  với mọi S' c T mà S' CÒN phủ hết   T thật sự tối tiểu
4  mọi i có beta_i < 0 đều không nằm trong T          không đưa đoạn gây hại vào đích
5  tổng_i credit(c_i) = 1                             theo cấu trúc
```

Bộ này **yếu hơn** bộ bất biến của bản Shapley (`Σ _c = F(E) - F(E')` là đẳng thức chính xác). Phép kiểm 1–3 chỉ xác nhận `T`, không xác nhận cách ta tìm ra nó. Đó là giá của việc bỏ completeness.

13.2 2 Chuẩn ngoài — HotpotQA

Mọi phép kiểm trên là **nội bộ**: `v(T)` `_q` do chính `v` đo, `_q` do chính `v_ref` đặt, `adq` do chính NLI của ta phán.

HotpotQA gán sẵn **2 đoạn chống đỡ** và tối đa **5 câu chống đỡ** — nhãn quy gán do người gán.

```
lấy ~500 truy vấn HotpotQA  ->  chạy TOÀN BỘ tầng A  ->  so T với 2 đoạn đã gán
giá:  500 x $0,37  =  $185

1  tỷ lệ ca mồi tham số THẬT     T chứa đoạn không nằm trong nguồn vàng
2  adq có đáng tin không          nó có đồng ý với nhãn người không
3  precision/recall của T         so với chuẩn ngoài
```

Con số 2 quan trọng nhất: đem `adq` loại nhãn trên 200.000 truy vấn mà chưa biết nó sai bao nhiêu thì chỉ là đổi một lỗi không biết sang một lỗi khác không biết.

MuSiQue không gán câu chống đỡ, nhưng gán **phân rã 1-hop kèm đáp án trung gian** — dùng nó kiểm chất lượng LLM chẻ câu hỏi.

Không cần lớp khớp chunk. `T` của ta là tập **tên tài liệu** (node `document`), và `supporting_documents` trong `train.json` của gfm-rag cũng là tập tên tài liệu — cùng không gian tên, so sánh trực tiếp bằng phép so tập.

Nhưng phải chạy trên split mà checkpoint CHƯA train. Loss giám sát của G-reasoner huấn luyện trên *"labeled relevant nodes"*, mà đó chính là `supporting_documents` — tức chính chuẩn ngoài này. Nên nếu hiệu chuẩn trên train split thì nhánh GFM của pool đã thấy sẵn nhãn vàng, và cả ba con số phồng giả. Dùng split đánh giá (1.000 mẫu mỗi bộ theo paper).

Và báo cáo mỗi con số **hai lần**: pool chỉ-BM25 (độc lập với nhãn vàng) và pool đầy đủ (nhánh GFM không độc lập). **Chênh lệch giữa hai lần chính là cỡ của rò rỉ.**

13.3 3 Phân tích độ nhạy

```
quét w_s, w_g, w_r mỗi cái ±50%        T đổi bao nhiêu lần?
quét tau_q quanh theta thuộc [0,90 ; 0,98]       W có ổn định không?
```

```
bootstrap g_theta, train lại                phương sai của credit
```

T lật lung tung khi nhiễu nhỏ thì **báo cáo cả biên Pareto** thay vì một T duy nhất.

---

14. Chạy theo lô

```
lô b    chạy 20.000 truy vấn bằng g_theta^(b)
        +- qua    ->  xuất T sang tầng B
        +- trượt  ->  hàng cứu
        lấy mẫu hàng cứu theo tầng  ->  chạy pipeline đầy đủ        <- ở ĐÂY
        gộp mọi nhãn mới  ->  train  g_theta^(b+1)
lô b+1  ...
```

Cứu phải xử lý **ở ranh giới lô**, cùng lúc với train lại, vì nhãn cứu là đầu vào giá trị nhất cho g_ ^(b+1).

```
lô    truy vấn   trượt sau thử lại   tỷ lệ cứu   số ca cứu
 1     20.000         ~9%   1.800        40%        720
 5     20.000         ~7%   1.400        25%        350
10     20.000         ~5%   1.000        10%        100
                                                   -----
                                                  ~3.000
```

Sàn 5% phải giữ, không bao giờ xuống 0 — nếu không thì ô «lỗi đồ thị» mất nguồn.
Và lý do chính của chia lô **không phải tiền**: mỗi ranh giới lô là một **cửa thoát**.

```
sau lô 1   p = 0,70 như dự  ->  chạy tiếp
           p = 0,45         ->  DỪNG. mới tốn ~$1.000, chưa tốn $12.500
           |T| phình đều    ->  dừng, sửa phần dư âm trước khi chạy tiếp
```

---

15. Sổ chi phí và sổ dữ liệu

```
                          nạp vào      chạy đủ    $/truy vấn       tổng
pha 1, pipeline đầy đủ     18.200       10.000      $0,37        $3.800
pha 2, đường rẻ           200.000      110.000      $0,037       $7.500
cứu, có phân tầng           3.000        3.000      $0,34        $1.000
hiệu chuẩn HotpotQA §12.2     500          500      $0,37          $185
                                                                ---------
                                                                $12.485

chạy pipeline đầy đủ cho cả 200.000                             $76.000
```

Khoảng **6 lần rẻ hơn**, cho ~104k nhãn **đáng tin**.

```
nguồn                            truy vấn    nhãn/truy vấn      tổng
pha 1, bốn vòng                   10.000         111         1.110.000
verification đường rẻ            110.000          10         1.100.000
```

```
cứu -> pipeline đầy đủ                3.000        111          333.000
ca trượt bị bỏ, không cứu            14.000          5           70.000
thoát ở A0/C0 (chỉ bảng phủ)         98.200          0                0
                                                              -----------
                                                               2.613.000
```

---

16. Chạy tay trọn vẹn một truy vấn — pha 2

Gom lại toàn bộ ví dụ thành một vết chạy, kèm token tiêu ở mỗi bước.

```
q = "ai đạo diễn phim thắng Oscar phim hay nhất 1994?"      y = "Robert Zemeckis"

bước  việc                                                    kết quả              token
-------------------------------------------------------------------------------------------
-     truy hồi 3 nguồn, rerank theo nguồn, gom 6 cụm          n = 40                   0
C0    chẻ câu hỏi                                             J = 2 mảnh             400
      bảng phủ, 40 x 2 = 80 lượt NLI                          mảnh 1 <- {3,7}          0
                                                              mảnh 2 <- {12,23,31}
      mọi mảnh có người chống đỡ  ->  KHÔNG thoát
      nghiệm phủ (giải chính xác)                             6 ứng viên cỡ 2          0
C1    v(rỗng)                                                    0,05                500
C2    g_theta chấm 40 đoạn đơn, v~_k che                            xếp hạng ứng viên        0
C3    v(R), R = top-2 mỗi cụm tôn trọng hạn ngạch             0,88                3.500
C4    v(C), cả 40 đoạn                                        0,81               10.500
      v_ref = max(0,88 ; 0,81) = 0,88     <- v(R) THẮNG v(C), đúng cảnh báo §5
      tau_q = 0,95 x (0,88 - 0,05) + 0,05 = 0,84
      v_ref - v(rỗng) = 0,83 >= 0,05  ->  KHÔNG thoát
C5    tầng 0: g_theta xếp {3,12} đầu bảng                        -                      0
C6    v({3,12})                                               0,94 >= 0,84          1.000
      adq({3,12}): cov = {1,2}                                đủ                      0
      tối tiểu - chỉ kiểm tập con CÒN phủ hết:
         cỡ 2 là cỡ tối tiểu của phủ, nên chỉ còn hai đơn tử
         v({3})                                               0,12 < 0,84           750
         v({12})                                              0,18 < 0,84           750
C7    nhánh (1)                                               T = {3, 12}
-------------------------------------------------------------------------------------------
                                                                              tổng  17.400
                                                                                   ~ $0,052
```

Ba chỗ cơ chế mới cứu được, nhìn rõ trong vết này:

```
1  đoạn 19 ("hiện tượng văn hóa") có v({19}) = 0,71 - cao hơn MỌI singleton hợp lệ.
   Bản chỉ-dùng-v sẽ coi nó là ứng viên mạnh và có thể đi theo nhánh đó.
   Bảng phủ loại nó khỏi danh sách ngay: cov({19}) = rỗng.

2  tối tiểu chỉ tốn 2 lượt (hai đơn tử), không phải 2^3 - 2 = 6.
   Vì {3} và {12} là các tập con DUY NHẤT có thể còn phủ hết - mà chúng không phủ hết,
   nên thật ra đã xong sau khi biết cỡ 2 là cỡ tối tiểu của phủ.

3  v(C) = 0,81 < v(R) = 0,88.  Nhồi 40 đoạn làm model phân tán.
   Nếu neo v_ref vào v(C) thì tau_q = 0,95x(0,81-0,05)+0,05 = 0,77 - THẤP hơn 0,07.
   Lúc đó v({19}) = 0,71 vẫn trượt, nhưng một tập mồi khác quanh 0,78 sẽ QUA.
```

Ca ngược — nếu pool thiếu: giả sử pool không có đoạn nào nói ai đạo diễn.

```
C0    bảng phủ:  mảnh 2 <- rỗng          ->  THOÁT NGAY
      T = rỗng,  cờ "pool thiếu"  ->  nuôi ô "lỗi đồ thị" §6.1
      tổng tiêu:  400 token  ~ $0,0012      (không phải 17.400)
```

---

17. Những con số chưa được kiểm chứng

```
p = 0,75                    GIẢ ĐỊNH. toàn bộ cột chi phí treo vào nó.
30% mỗi lần thử lại         GIẢ ĐỊNH. thấp hơn thì p hiệu dụng 0,91 không đạt.
45% thoát ở A0/C0           suy từ MuSiQue 46,2% / HotpotQA 44,6%, CHƯA đo trên corpus riêng.
|T| = 2-3 là cột chính      theo lý thuyết 2-3 hop, chưa đo.
adq sai bao nhiêu           CHƯA BIẾT. §12.2 tồn tại để đo đúng chỗ này.
```

Hai cổng miễn phí phải chạy **trước** khi cam kết ngân sách:

```
cổng p       500 truy vấn pha 1 giữ lại, chạy cả đường pha 2       0 lượt LLM
chuẩn ngoài  500 truy vấn HotpotQA, so với nguồn vàng              $185
```

---

18. Tham chiếu

1. Joren et al., *Sufficient Context: A New Lens on Retrieval Augmented Generation Systems*,
ICLR 2025. arXiv:2411.06037 — định nghĩa đủ-ngữ-cảnh không dùng đáp án vàng; autorater 93%; MuSiQue-Ans 46,2% và HotpotQA 44,6% ca có ngữ cảnh đủ; model trả lời đúng 35–62% số ca ngữ cảnh không đủ. **2.** *Relevance Is Not Sufficient Evidence: Detecting Evidence Gaps Before Generation in RAG*,
arXiv:2609.37469 — RINSE, 0,837 pairwise; 12 model vẫn trả lời 40–99% ca thiếu bằng chứng dù được yêu cầu từ chối. **3.** *Verification Without Sufficiency: Per-Chunk Filtering Fails on Multi-Hop RAG, and
Decomposition Repairs It*, arXiv:2608.00585 — kiểm từng đoạn đối câu hỏi gốc thất bại ở multi-hop; phân rã sửa được. **Chỉ lấy cơ chế, chưa trích số. 4.** Yang et al., *HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering*,
EMNLP 2018 — 2 đoạn chống đỡ, tối đa 5 câu chống đỡ, gán ở mức câu. **5.** Deegan & Packel, *A new index of power for simple n-person games*, Int. J. Game Theory 7(2),
1978. **Bạn phải xác thực trích dẫn này. 6.** Banzhaf, *Weighted voting doesn't work*, Rutgers Law Review 19, 1965. **Phải xác thực.**
7. Holler & Packel, *Power, luck and the right index*, J. Economics 43(1), 1983.
Phải xác thực.